

University of Tehran
Faculty of Electrical and Computer Engineering
Assignment 5: Neural Networks and Deep Learning

Instructors: Dr. Mohammad Gorji, Aryan Firouzi

Deadline: Day 16, 1404 (Approx. Jan 6, 2026)

Topics: Image Re-identification (Transformers) & Large Language Diffusion Models

Question 1: Image Re-identification using Transformers (100 Points + Bonus)

In this exercise, we will use Transformers for the task of Image Re-identification (Re-ID). Unlike traditional classification (e.g., distinguishing a cat from a dog), Re-ID involves identifying a specific instance (e.g., a specific person) across different camera views. This requires the network to learn fine-grained details (like tattoos, logos, or specific clothing patterns) rather than just general class structures.

We will compare two common architectures: **ResNet (CNN)** and **BotNet (Transformer-based)**.

1.1 Data Preparation (10 Points)

- **Download:** Download the dataset from the provided link.
- **Preprocessing:** Explain what preprocessing steps are useful for this dataset (e.g., resizing, normalization). Since we have fewer than 5 images per class, aggressive augmentation is necessary to increase accuracy. Explain which augmentation methods you used.
- **Splitting:** Split the dataset into **Train** and **Test** sets. You can use an 80/20 split.
- **Graph Sampling:** The text mentions using a “Graph Sampling” method. Explain how this sampling affects the data distribution.
- **Batch Size:** Use a batch size of 32 (or a reasonable number your hardware can handle).
- **Visualization:** Display a few sample images from the dataset after preprocessing.

1.2 ResNet Model Training (10 Points)

- Train a **ResNet50** model on the dataset.
- **Configuration:** Train for at least **20 epochs**. Use Cross-Entropy Loss.
- **Reporting:**
 - Plot the Training Loss and Test Loss per epoch.
 - Report the final accuracy/error rate on the Test set.
 - There is no need to design the ResNet architecture from scratch; you may use existing libraries, but you must modify the output layer to match the number of classes.

1.3 BotNet Model Training (40 Points)

- Implement and train a **BotNet** (Bottleneck Transformer) model.
- **Architecture:** BotNet replaces the spatial convolutions in the final stage (e.g., the last 3 layers of a ResNet) with **Multi-Head Self-Attention (MHSA)**.
- **Implementation:** Design the architecture such that the feature map dimensions are preserved while incorporating the attention mechanism.
- **Training:** Train this model using the same hyperparameters and data split as the ResNet model for a fair comparison.
- **Reporting:** Plot loss curves and report final accuracy.
- *Note: Training might be slow (approx. 3–5 minutes per epoch on a T4 GPU). Ensure you use a GPU.*

1.4 Results Analysis (15 Points)

- Compare the performance of ResNet and BotNet.
- Which model performed better? Why?
- Did the Transformer-based model (BotNet) perform better on specific classes or difficult cases compared to the CNN (ResNet)?
- Analyze if there is a significant difference in convergence speed or stability.

1.5 Attention Map Visualization & Analysis (15 Points)

- Extract the output of the **Attention Layer** from the trained BotNet.
- Visualize these outputs as **Heatmaps** overlaid on the original images.
- **Analysis:**
 - Does the model focus on the correct features (e.g., the person/object)?

- Is the model “cheating” by focusing on background information or irrelevant patterns?
- Compare the heatmaps of correct predictions vs. incorrect predictions.

1.6 Counterfactual Attention (Bonus: 5 Points)

- Implement **Counterfactual Attention** training.
- **Method:**
 1. Get the standard output features from the model.
 2. Mask the features based on their attention weights (masking the most attended/important features).
 3. Calculate the entropy difference between the original output and the masked output.
 4. Retrain/Fine-tune the model to maximize the entropy difference (forcing the model to look at other useful features, not just the most obvious ones).
- Repeat the analysis from section 1.5 with this new model. Did the heatmaps change?

Question 2: Large Language Diffusion Models (100 Points + Bonus)

This question focuses on **LLaDA (Large Language Diffusion with Autoregression)**. Unlike standard Autoregressive (AR) models that predict the next token t based on $t - 1$, LLaDA treats text generation as a **Masked Iterative Generation** process. We will use this to build a **Text-to-SQL** pipeline.

2.1 Theoretical Questions (30 Points)

Answer based on the LLaDA paper/theory:

1. **Difference between Methodologies:** Explain the fundamental difference between Autoregressive generation and Masked Iterative Generation. How does the generation process look different to the user?
2. **Forward Masking as Noise:** Explain how “Forward Masking” acts as a **Noise Model**. How does masking tokens relate to adding noise in continuous diffusion models?
3. **Reweighting Loss:** Why do we need to **reweight** the loss function? (Hint: Information density varies when masking 15% vs. 90% of tokens).

2.2 Data, Prompt, and Evaluation Tools (20 Points)

Dataset: Use `gretelai/synthetic_text_to_sql`.

2.2.1 Pipeline Construction

- Load the dataset and create Train/Validation splits.
- **Stats:** Report average schema and question lengths.
- **Filtering:** Implement a strategy for long schemas (e.g., truncation or filtering). Explain your strategy in the report.
- **Requirements:** Use at least 1000 training samples and 300 test samples.

2.2.2 Chat Format Prompt Design

- Design a prompt template using `tokenizer.apply_chat_template`.
- **Structure:** System (Instructions), User (Schema + Question), Assistant (Gold SQL).
- **Crucial:** Identify exactly where the “Assistant” response begins (apply noise only to the answer).

```
1 SYSTEM_PROMPT = "You are a Text-to-SQL assistant. Output ONLY the SQL\n    query."\n2 user_content = "Schema:\\n...\\nQuestion:\\n..."\n3 messages = [\n4     {"role": "system", "content": SYSTEM_PROMPT},\n5     {"role": "user", "content": user_content},\n6     {"role": "assistant", "content": "SELECT * FROM table;"}\n7 ]\n8 # Use tokenizer to apply template
```

Listing 1: Prompt Construction Example

2.2.3 SQL Normalization & Metrics

- Implement **SQL Normalization**: Lowercase, remove backticks/quotes, standardize spaces, remove trailing semicolons.
- Implement **Exact Match (EM)** and **Normalized Exact Match**.

2.3 Model Loading and Training (30 Points)

2.3.1 Model Loading & Concepts

- Load GSAI-ML/LLaDA-8B-Instruct.
- **Quantization**: Use 4-bit quantization. Explain why it is used.
- **LoRA**: Use LoRA for fine-tuning. Explain why.
- **Settings**: Set use_cache=False. Ensure mask_token_id is set correctly.

2.3.2 Forward Masking Implementation

Implement `noisy_batch(input_ids)`:

1. Sample random time $t \in [0, 1]$.
2. Determine probability p_{mask} based on t .
3. **Masking**: Randomly replace tokens in the **Answer** part with [MASK]. Do not mask the Prompt.
4. **Output**: Return masked IDs, boolean mask of masked indices, and p_{mask} .

2.3.3 Training Loop

- Calculate Cross-Entropy Loss **only** on masked tokens.
- **Reweight**: Apply reweighting based on p_{mask} .
- Handle edge cases (e.g., empty masks) to prevent NaNs.

2.4 Evaluation and Block Diffusion Sampling (20 Points)

2.4.1 Block Diffusion Sampling

Implement the generation strategy:

1. Start with a fully masked response.
2. **Iterative Process:** Predict logits $\rightarrow$ Select a “Block” of high-confidence tokens to commit $\rightarrow$ Keep others masked $\rightarrow$ Repeat.
3. Explain the trade-off of Block Sampling.

2.4.2 Post-processing & Error Analysis

- Extract SQL, clean output, and evaluate on Test set (> 200 samples).
- Report Normalized EM and show 10 sample outputs.

2.4.3 Bonus Idea (5 Points)

Propose a creative improvement for LLaDA in Text-to-SQL (e.g., noise schedule, sampling strategy) and explain the logic.